



Snowflake 

HANDS-ON PROJECT



Hotel Booking System



Snowflake Hands-on Project

Hotel Booking Analytics

PROJECT SCENARIO

You have joined a hotel chain as a **Data Engineer**.

The hotel stores booking information in CSV files. Unfortunately, the data is messy.

Management wants clean, reliable data for reporting and analytics.

Your task is to build a complete data pipeline inside Snowflake.

Skills Covered

- ✓ Database Creation
- ✓ Schema Creation
- ✓ Warehouse
- ✓ File Format
- ✓ Stage
- ✓ Loading CSV
- ✓ COPY INTO
- ✓ Data Cleaning
- ✓ SQL Functions
- ✓ CASE
- ✓ DATE Functions
- ✓ Aggregate Functions
- ✓ Views

✓ KPI Queries

✓ Snowsight Dashboard

✖ **Project Background**

The hotel currently has:

✖ Missing values

✖ Invalid emails

✖ Incorrect booking statuses

✖ Negative revenue

✖ Dates stored as text

✖ Duplicate formatting

Management needs:

📊 Monthly Revenue

📊 Monthly Bookings

🏨 Top Revenue Cities

📁 Booking by Room Type

📋 Booking Status Analysis

📊 KPI Dashboard

📁 **Project Architecture**

CSV File

↓

Snowflake Stage

↓

Raw Table



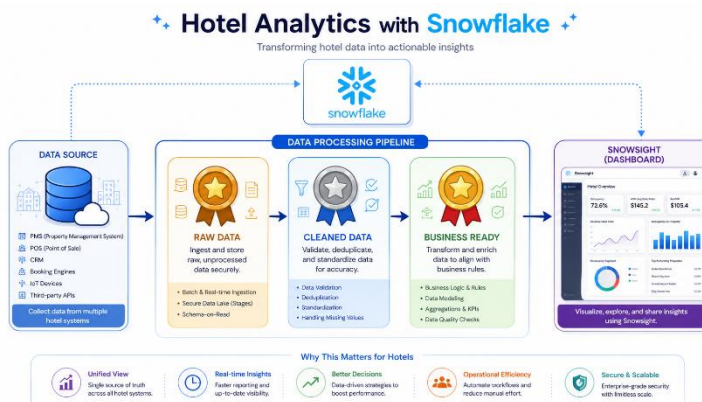
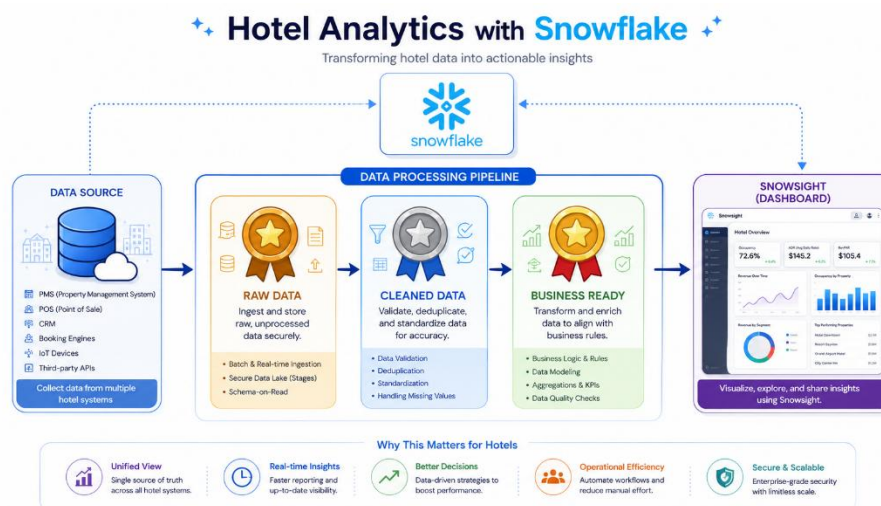
Cleaned Table

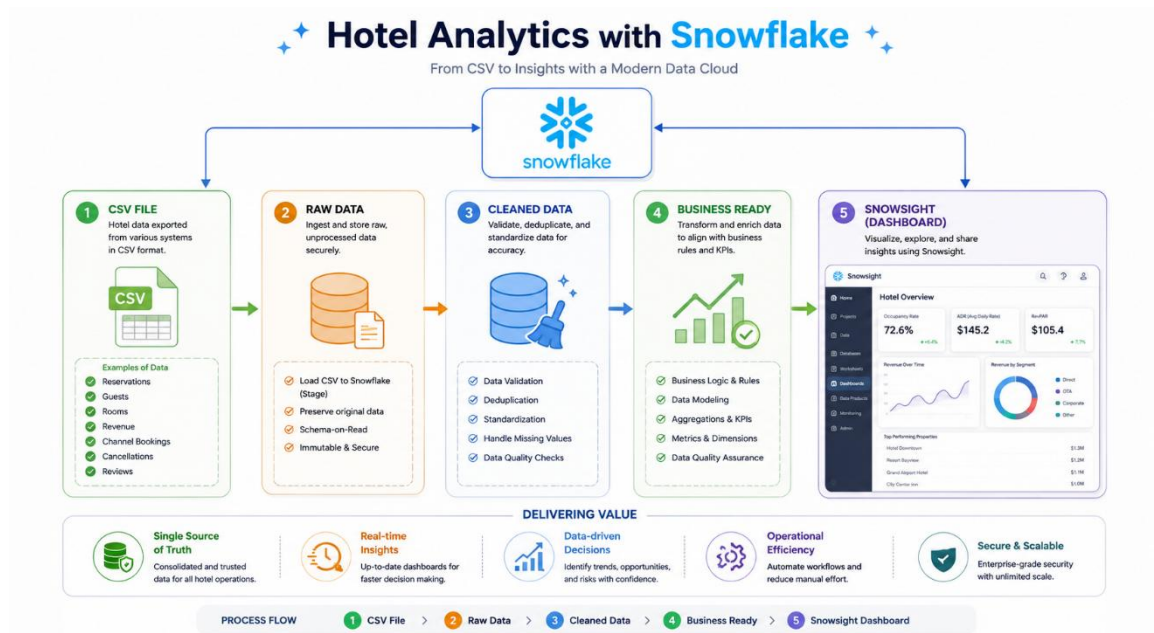


Analytics Views



Snowsight Dashboard





3 **Bronze layer**: Raw data ingestion

2 **Silver layer**: Data validation, cleansing, and standardization

1 **Gold layer**: Business-ready dimensional and summary datasets

Snowsight dashboards for analytics

Dataset Columns

Students receive

```
booking_id
customer_name
booking_date
check_in
check_out
city
room_type
booking_status
price_per_night
number_of_nights
```

Environment Setup

Create the Snowflake environment.

✓ Task 1

Create a warehouse.

Expected name

HOTEL_WH

Answer

```
CREATE OR REPLACE WAREHOUSE HOTEL_WH  
WITH  
WAREHOUSE_SIZE='XSMALL'  
AUTO_SUSPEND=60  
AUTO_RESUME=TRUE;  
  
USE WAREHOUSE HOTEL_WH;
```

✓ Task 2

Create a database.

Expected name

HOTEL_DB

Answer

```
CREATE DATABASE HOTEL_DB;  
  
USE DATABASE HOTEL_DB;
```

✓ Task 3

Create a schema

BOOKINGS

Answer

```
CREATE SCHEMA BOOKINGS;
```

```
USE SCHEMA BOOKINGS;
```

Load CSV

✓ Task

 Create a CSV File Format.

Explanation

This tells Snowflake:

- ✓ Ignore first row
- ✓ Handle commas correctly
- ✓ Convert NULL values

Answer

```
-- Create File Format
CREATE OR REPLACE FILE FORMAT FF_CSV
  TYPE = 'CSV'
  FIELD_OPTIONALLY_ENCLOSED_BY = '"'
  SKIP_HEADER = 1
  NULL_IF = ('NULL', 'null', '')
```

✓ Task

 Create an internal stage.

Answer

```
-- Create Stage
CREATE OR REPLACE STAGE STG_HOTEL_BOOKINGS
  FILE_FORMAT = (FORMAT_NAME = FF_CSV)
  ON_ERROR = 'CONTINUE';
  FILE_FORMAT = FF_CSV;
```

✓ Task

Upload the CSV Dataset

Students

- Open Stage
- Upload hotel_bookings.csv

(No SQL required.)

Bronze Layer

Objective

Store raw data exactly as received.

Task

Create Raw Table

Answer

```
-- Create Table BRONZE_HOTEL_BOOKING
CREATE TABLE BRONZE_HOTEL_BOOKING (
  booking_id STRING,
  hotel_id STRING,
  hotel_city STRING,
  customer_id STRING,
  customer_name STRING,
  customer_email STRING,
  check_in_date STRING,
  check_out_date STRING,
  room_type STRING,
  num_guests STRING,
  total_amount STRING,
  currency STRING,
  booking_status STRING
);
```

Task

Load CSV Data

Answer

```
-- Loading Data from Stage to Bronze Table
COPY INTO BRONZE_HOTEL_BOOKING
FROM @STG_HOTEL_BOOKINGS
FILE_FORMAT = (FORMAT_NAME = FF_CSV)
```

```
ON_ERROR = 'CONTINUE';
```

Task

Preview data.

Answer

```
SELECT * FROM BRONZE_HOTEL_BOOKING LIMIT 50;
```

Expected Output

Students should see all booking records.

No cleaning yet.

Silver Layer

Objective

Clean the raw data.

Task

Create Table SILVER_HOTEL_BOOKINGS

Answer

```
CREATE TABLE SILVER_HOTEL_BOOKINGS (  
  booking_id VARCHAR,  
  hotel_id VARCHAR,  
  hotel_city VARCHAR,  
  customer_id VARCHAR,  
  customer_name VARCHAR,  
  customer_email VARCHAR,  
  check_in_date DATE,  
  check_out_date DATE,  
  room_type VARCHAR,  
  num_guests INTEGER,  
  total_amount FLOAT,  
  currency VARCHAR,
```

```
        booking_status VARCHAR
    );
```

Checking for errors

```
SELECT customer_email
FROM BRONZE_HOTEL_BOOKING
WHERE NOT (customer_email LIKE '%@%.%')
       OR customer_email IS NULL

SELECT total_amount
FROM BRONZE_HOTEL_BOOKING
WHERE TRY_TO_NUMBER(total_amount) < 0;

SELECT check_in_date, check_out_date
FROM BRONZE_HOTEL_BOOKING
WHERE TRY_TO_DATE(check_out_date) < TRY_TO_DATE(check_in_date);

SELECT DISTINCT booking_status
FROM BRONZE_HOTEL_BOOKING;
```

Step 1 — Find Invalid Emails

Run

```
SELECT customer_email
FROM BRONZE_HOTEL_BOOKING
WHERE NOT(customer_email LIKE '%@%.%')
OR customer_email IS NULL;
```

Question

How many invalid emails exist?

Step 2 — Find Negative Revenue

```
SELECT total_amount
FROM BRONZE_HOTEL_BOOKING
WHERE TRY_TO_NUMBER(total_amount) < 0;
```

Question

Why shouldn't hotel revenue be negative?

Step 3 — Find Invalid Dates

```
SELECT check_in_date,
       check_out_date
```

```
FROM BRONZE_HOTEL_BOOKING
WHERE TRY_TO_DATE(check_out_date)
<
TRY_TO_DATE(check_in_date);
```

? Question

Why are these records invalid?

🔍 Step 4 — Find Incorrect Booking Status

```
SELECT DISTINCT booking_status
FROM BRONZE_HOTEL_BOOKING;
```

Students should notice values like

confirmeeed

confirmd

Confirmed

🔧 Step 5 — Clean Data

Run

✅ Task

Inserting Cleaned data to Silver layer

```
INSERT INTO SILVER_HOTEL_BOOKINGS
SELECT
    booking_id,
    hotel_id,
    INITCAP(TRIM(hotel_city)) AS hotel_city,
    customer_id,
    INITCAP(TRIM(customer_name)) AS customer_name,
    CASE
        WHEN customer_email LIKE '%@%.%' THEN LOWER(TRIM(customer_email))
        ELSE NULL
    END AS customer_email,
    TRY_TO_DATE(NULLIF(check_in_date, '')) AS check_in_date,
    TRY_TO_DATE(NULLIF(check_out_date, '')) AS check_out_date,
    room_type,
    num_guests,
    ABS(TRY_TO_NUMBER(total_amount)) AS total_amount,
    currency,
```

```

CASE
    WHEN LOWER(booking_status) in ('confirmeed', 'confirmd') THEN 'Confirmed'
    ELSE booking_status
END AS booking_status
FROM BRONZE_HOTEL_BOOKING
WHERE
    TRY_TO_DATE(check_in_date) IS NOT NULL
    AND TRY_TO_DATE(check_out_date) IS NOT NULL
    AND TRY_TO_DATE(check_out_date) >= TRY_TO_DATE(check_in_date);

```

What this SQL does

-  Trims spaces
-  Converts text to dates
-  Cleans email addresses
-  Removes negative revenue
-  Fixes city names
- ✓ Corrects booking status
- ✗ Removes invalid dates

Task

Preview data.

Answer

```
SELECT * FROM SILVER_HOTEL_BOOKINGS LIMIT 30;
```

Gold Layer

This layer is used directly by dashboards.

Daily Revenue

```
CREATE TABLE GOLD_AGG_DAILY_BOOKING AS

SELECT

check_in_date AS date,

COUNT(*) AS total_booking,

SUM(total_amount) total_revenue

FROM SILVER_HOTEL_BOOKINGS

GROUP BY check_in_date

ORDER BY date;
```

Revenue by City

```
CREATE TABLE GOLD_AGG_HOTEL_CITY_SALES AS

SELECT

hotel_city,

SUM(total_amount) total_revenue

FROM SILVER_HOTEL_BOOKINGS

GROUP BY hotel_city

ORDER BY total_revenue DESC;
```

Clean Booking Table

```
CREATE TABLE GOLD_BOOKING_CLEAN AS

SELECT *

FROM SILVER_HOTEL_BOOKINGS;
```

Task

Preview data.

```
SELECT * FROM GOLD_AGG_DAILY_BOOKING LIMIT 30;
```

```
SELECT * FROM GOLD_AGG_HOTEL_CITY_SALES LIMIT 30;
```

Dashboard SQL

KPI 1

Average Booking Value

```
SELECT AVG(total_amount)
FROM GOLD_BOOKING_CLEAN;
```

KPI 2

Total Guests

```
SELECT SUM(num_guests)
FROM GOLD_BOOKING_CLEAN;
```

KPI 3

Total Bookings

```
SELECT COUNT(*)
FROM GOLD_BOOKING_CLEAN;
```

KPI 4

Total Revenue

```
SELECT SUM(total_amount)
FROM GOLD_BOOKING_CLEAN;
```

Monthly Revenue

```
SELECT
```

```
date,  
total_revenue  
FROM GOLD_AGG_DAILY_BOOKING  
ORDER BY date;
```



Monthly Bookings

```
SELECT  
date,  
total_booking  
FROM GOLD_AGG_DAILY_BOOKING  
ORDER BY date;
```



Top Revenue Cities

```
SELECT  
hotel_city,  
total_revenue  
FROM GOLD_AGG_HOTEL_CITY_SALES  
ORDER BY total_revenue DESC  
LIMIT 5;
```



Booking Status

```
SELECT  
  
booking_status,  
  
COUNT(*) total  
  
FROM GOLD_BOOKING_CLEAN  
  
GROUP BY booking_status;
```

👉 Booking by Room Type

```
SELECT  
  
room_type,  
  
COUNT(*) total_bookings  
  
FROM GOLD_BOOKING_CLEAN  
  
GROUP BY room_type  
  
ORDER BY total_bookings DESC;
```

KPI Queries

Students create KPI cards in Snowsight.

Total Revenue

```
SELECT SUM(total_amount)
FROM CLEAN_BOOKINGS;
```

Total Bookings

```
SELECT COUNT(*)
FROM CLEAN_BOOKINGS;
```

Total Customers

```
SELECT COUNT(DISTINCT customer_id)
FROM CLEAN_BOOKINGS;
```

Total Hotels

```
SELECT COUNT(DISTINCT hotel_id)
FROM CLEAN_BOOKINGS;
```

Total Cities

```
SELECT COUNT(DISTINCT hotel_city)
FROM CLEAN_BOOKINGS;
```

Average Guests

```
SELECT AVG(num_guests)
FROM CLEAN_BOOKINGS;
```

Learning Outcomes

By completing this project, students will be able to:

-  Create Snowflake databases, schemas, and warehouses.
-  Load CSV data using stages and the COPY INTO command.
-  Clean and standardize raw datasets with SQL.
-  Create calculated columns and perform aggregations.
-  Build reusable SQL views for analytics.
-  Design interactive dashboards in Snowsight with KPI cards and charts.
-  Apply window functions for ranking and growth analysis.
-  Deliver an end-to-end analytics solution similar to a real-world data engineering workflow.